Neural Network for Mushroom Classification

Noah Vlodek

December 2024

Contents

1 Abstract **3**

2 Introduction **4**
2.1 Problem 4
2.2 Why I chose this project 4

3 Phase 1: Data Analysis and Preparation **4**
3.1 Summary Statistics of Numerical Features 5
3.2 Normalization of Numerical Features 5
3.3 Encoding Categorical Features 6
3.3.1 Distribution of Select Categorical Features 7
3.4 Distribution of the Output Variable 9

4 Phase 3: Model Selection and Evaluation **9**
4.1 Parameters 9
4.2 Best Model and Summary 9
4.3 Learning Curves of Models 10
4.4 RUC/AUC of the best Model 11

5 Phase 4: Feature Importance **12**
5.1 Running Model for Individual Features 12
5.2 Iteratively Removing Un-important Features 14

6 Conclusion **15**

List of Tables

1 Dropped Columns 4
2 Summary Statistics 5
3 Handling Missing Data 7
4 Categorical Features 7
5 Model Accuracies 10
6 Other Metrics 10
7 Feature mapping by indices 13

List of Figures

1 Cap-Diameter Normalized 5
2 Stem-Height Normalized 6
3 Stem-Width Normalized 6
4 Distribution of Cap Color 7
5 Distribution of Gill attach 8
6 Distribution of Habitat 8
7 Distribution of Season 8
8 Output Class Distribution 9
9 Logistic Learning Curve 10
10 2-1 Neural Network Learning Curve 11
11 4-2-1 Neural Network Learning Curve 11
12 AUC and RUC Curves of Best Model 12
13 Val Accuracy Individual Features 13
14 Val Accuracy After Feature Removal 14

1 Abstract

The dataset I chose for my project is the Secondary Mushroom dataset, available on the UCI Machine Learning Repository website. The world of mushrooms is fascinating because based on twenty rather complex features, one can predict whether a mushroom is poisonous or edible. In the pre-processing phase, I checked for missing values, one-hot encoded categorical features, and normalized continuous numerical features.

The objective of this project is to build a neural network to classify a given mushroom as poisonous or edible. Based on twenty different features in the data, the goal was to slowly increase the size of the neural network until the model performed well in the validation set. The best performing model was one with 4 neurons in the first layer, 2 neurons in the second layer, and 1 neuron in the third layer, which predicted mushroom classifications with an accuracy of 94.60% on the validation set. The precision, f1 score, and recall were also very good for this model. The final model ended up as almost a perfect fit for the validation set.

After finding the best model, I ran a model of the individual features, as an attempt to find the most important feature. The next part of the analysis was to iteratively remove features, starting with the most unimportant feature, which yielded an interesting result: as I removed the features, the validation accuracy kept falling until it was near the random baseline accuracy. The process exemplified that there were no features that were clearly correlated with the edibility of a mushroom.

2 Introduction

2.1 Problem

The problem I am trying to solve by building a neural network is a binary classification problem: whether or not a mushroom is edible. Predicting whether a mushroom is edible is difficult because edibility is dependent on many factors, such as the size of the mushroom, the color of the mushroom, and the habitat around the plant. In this project, my goal is to build a neural network model to accurately predict whether or not a mushroom is edible.

2.2 Why I chose this project

I chose this project because the mushrooms are very interesting to me. It is cool that these fungi have so many different features, like cap shapes, stem heights and color of the mushroom gills. The world of mushrooms is interesting because there is a complex relationship between the 20 features in the data that together, can tell one if the mushroom is edible or poisonous.

3 Phase 1: Data Analysis and Preparation

The data set that I chose for my project is 'Secondary Mushroom', available from the UCI Machine Learning Repository (`https://archive.ics.uci.edu/dataset/848/secondary+mushroom+dataset`). I chose this data set because it seemed interesting to be able to tell whether a mushroom is poisoned or not. Another reason why I picked this dataset is that the classes are more balanced than any other dataset I tried to use. There are twenty features and one target class. I was able to reduce the number of features to 13, by dropping 7 of them. I dropped these 7 because of the number of missing values in these columns. My threshold was that if more than 20% of the data are missing, I drop the column.

Feature	Percentage of Values Missing
Cap Surface	23.12%
Gill Spacing	41.04%
Stem Root	84.39%
Stem Surface	62.43%
Veil Type	94.80%
Veil Color	87.86%
Spore Print Color	89.60%

Table 1: Dropped Columns

3.1 Summary Statistics of Numerical Features

Feature	Count	Mean	Stand Dev	Min	25%	50%	75%	Max
Cap-Dimeter	61,069	6.734	5.265	0.380	3.480	5.860	8.540	62.340
Stem-Height	61,069	6.582	3.370	0	4.640	5.950	7.740	33.920
Stem-Width	61,069	12.149	10.036	0	5.210	10.190	16.570	103.910

Table 2: Summary Statistics

3.2 Normalization of Numerical Features

To normalize the numerical features, I used the Z-score method. I used the following formula on each column:

$$Normalized_i = \frac{Col_i - \mu_{col}}{\sigma_{col}} \tag{1}$$

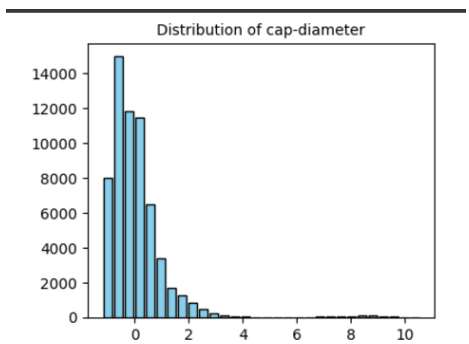

Figure 1: Cap-Diameter Normalized

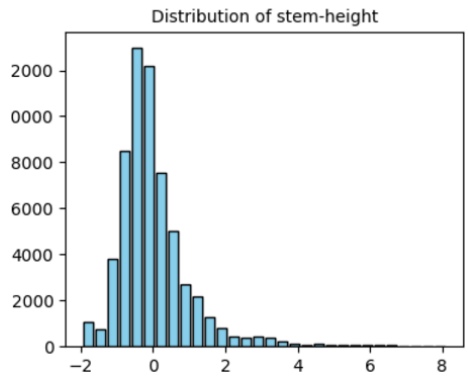

Figure 2: Stem-Height Normalized

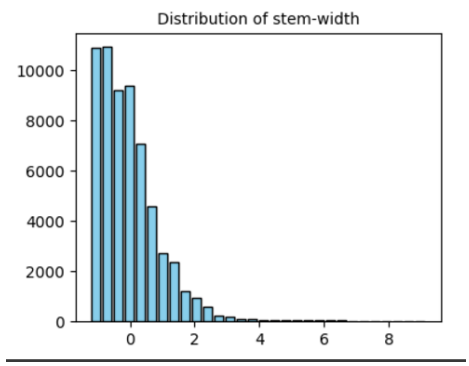

Figure 3: Stem-Width Normalized

3.3 Encoding Categorical Features

To preprocess the categorical features, I used the getdummies function available in the pandas library. I did this because a neural network model, if I had used label encoding (0,1,2,3,..,etc.) the model may see 3 as more desirable than a 0. The process this function uses is one-hot encoding, which creates a separate binary column for each category in the data (1 for true or 0 for false).

Feature	How Missing Data was Handled	Why
Gill Attachment	Filled the 16% Missing Data with 'f' (none)	Mode
Ring Type	Filled the 4% Missing Data with 'f' (none)	Mode

Table 3: Handling Missing Data

Here's a table summarizing the categorical features:

Feature	Definition
Cap Shape	Shape of upper most part of mushroom
Cap Surface	texture or pattern of a mushroom's cap
Cap Color	color of a mushroom's cap
Bruise or Bleed	whether the the mushroom bruises or bleed
Gill attach	how the gills of mushroom are connected to the stem
Gill Spacing	how close each gill is to one another
Gill Color	color of the gill
Stem Root	root type of mushroom
Stem Surface, Color	pattern, color of mushroom's stem
Ring Type, Has a Ring	ring type of the stalk of mushroom, and if it has a ring
Habitat	environment where mushroom grows
Season	season of the year

Table 4: Categorical Features

3.3.1 Distribution of Select Categorical Features

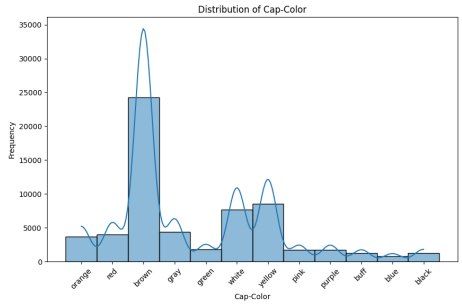

Figure 4: Distribution of Cap Color

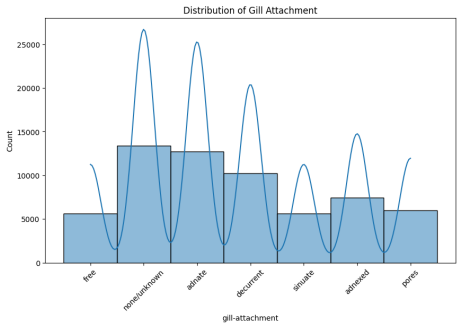

Figure 5: Distribution of Gill attach

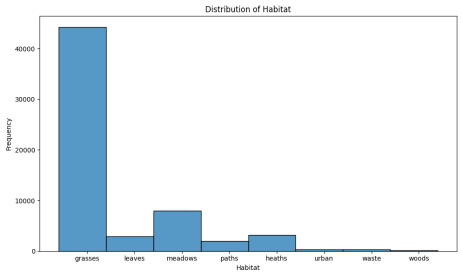

Figure 6: Distribution of Habitat

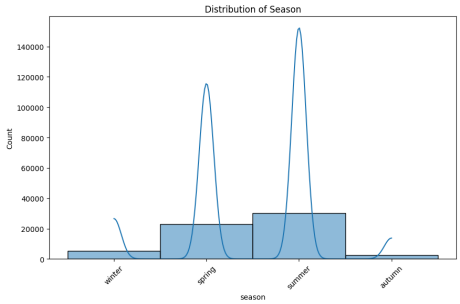

Figure 7: Distribution of Season

3.4 Distribution of the Output Variable

The output variable 'Class' is binary: whether the mushroom is edible or not. The class distribution is about 55-45 poisonous.

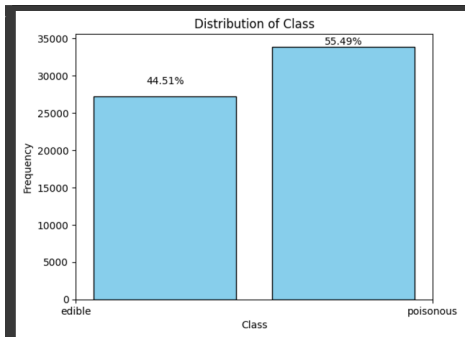

Figure 8: Output Class Distribution

4 Phase 3: Model Selection and Evaluation

For this phase of the project, my goal was to produce the best accuracy for my validation set. To begin this phase, I first shuffled the rows of the Mushroom data set before splitting into training and validation sets, using an 80-20 split. I started with a random baseline classifier and gradually increased the model. I began with a logistic regression model and gradually added more layers until the validation accuracy was the best it could be. I ended up running four models in all: the random baseline, logistic regression, and two neural networks: 4-2-1 and 2-1.

4.1 Parameters

The parameters I chose to use for my models were 512 for the batch-size parameter, 1000 for the epochs, and used two callbacks for each model: (1) Early Stopping and (2) Model Check-pointing. If my model did not improve after 20 epochs through the training set, I stopped the model.

4.2 Best Model and Summary

After running these four models, it is clear that my best model is the neural network model that has 4-neurons in the first layer, 2-neurons in the second layer and 1-neuron in the third layer.

Model	Acc. on Training Set	Acc. on Validation Set
Random baseline classifier	55.38%	55.95%
Logistic Regression	76.61%	76.97%
Neural Network Model (2-1)	83.25%	83.03%
Neural Network Model (4-2-1)	94.55%	94.60%

Table 5: Model Accuracies

Model	F1	Precision	Recall
Random baseline classifier	0.72	55.95%	100.00%
Logistic Regression	0.79	78.65%	78.88%
Neural Network Model (2-1)	0.84	88.95%	79.56%
Neural Network Model (4-2-1)	0.95	91.67%	99.39%

Table 6: Other Metrics

An interesting part about these metrics is that these increase slightly from the random baseline to logistic to the 2-1 neural network, but the metrics take a huge just from the 2-1 to the 4-2-1 neural networks.

4.3 Learning Curves of Models

Here are the learning curves for each of my models: **Logistic Regression**

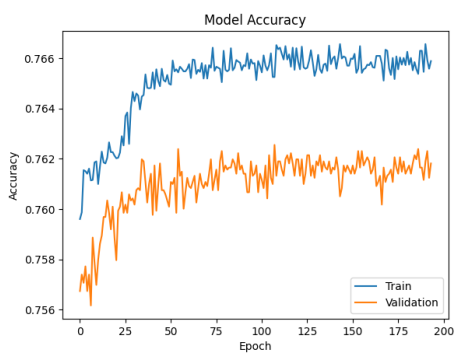

Figure 9: Logistic Learning Curve

2-1 Neural Network Curve

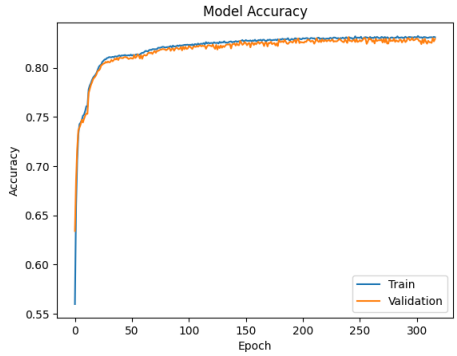

Figure 10: 2-1 Neural Network Learning Curve

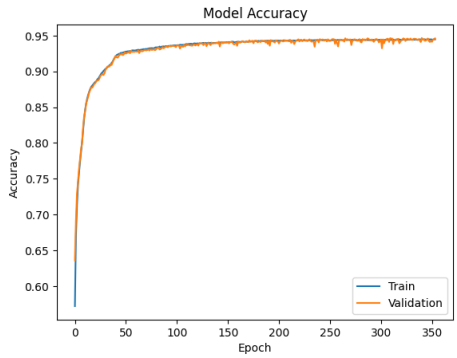

Figure 11: 4-2-1 Neural Network Learning Curve

4.4 RUC/AUC of the best Model

So, my final model was four neurons, two neurons, and 1 neuron in their layers. Here is the RUC/AUC curve of the best model:

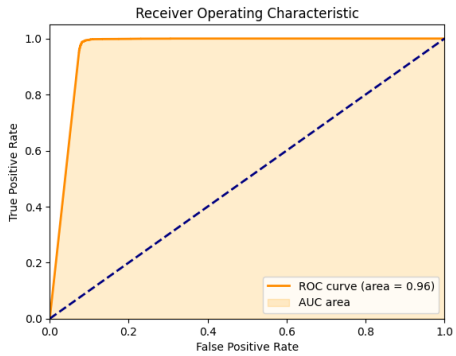

Figure 12: AUC and RUC Curves of Best Model

5 Phase 4: Feature Importance

In this phase, there are two main tasks. In the first task, I have to run the best model (the 4-2-1 neural network model) for each individual feature. I then have to see the validation accuracies for each of these individual models. This will help for the second task, because I must iteratively remove features, starting with the most unimportant feature, aka the feature that produces the lowest validation accuracy.

5.1 Running Model for Individual Features

For this part of the phase, I ran the code for all 76 features in the data. The most interesting part about the phase is that the validation accuracy is about the same for each of the features. The range on the validation accuracies is from about 55% to 59%. Two of the most important features, according to my model, are stem height and stem width.

Index	Feature	Index	Feature	Index	Feature
0	No Stem Color	**26**	Leafy Habitat	**51**	Flat Cap
1	White Stem	**27**	Boolean Bruise/Bleed	**52**	Conical Cap
2	Purple Stem	**28**	Grassy Habitat	**53**	No gill attach
3	Green Stem	**29**	Zone Ring	**54**	Bell Cap
4	Pink Stem	**30**	Flaring Ring	**55**	has-ring
5	Orange Stem	**31**	Pendant Ring	**56**	Summer
6	Brown Stem	**32**	Movable Ring	**57**	Sinuate Gill attach
7	Blue Stem	**33**	No ring	**58**	Annexed Gill
8	Black Stem	**34**	Evanescent Ring	**59**	Buff Color Ring
9	Yellow Stem	**35**	Yellow Cap	**60**	Red Gill
10	Red Stem	**36**	Convex Cap	**61**	No Gill Color
11	Yellow Gill	**37**	Sunken Cap	**62**	Grooved Ring
12	Purple Gill	**38**	Red Cap	**63**	Brown Cap
13	Green Gill	**39**	Gray Cap	**64**	Buff Color Stem
14	Pink Gill	**40**	cap-diameter	**65**	Spring Season
15	Orange Gill	**41**	Black Cap	**66**	Large Ring
16	Brown Gill	**42**	Blue Cap	**67**	Spherical Cap
17	Black Gill	**43**	Gray Gill	**68**	Gray Stem
18	Heath Habitat (shrubland)	**44**	Orange Cap	**69**	Buff Color Cap
19	Autumn Season	**45**	Pink Cap	**70**	Winter Season
20	Other Cap Shape	**46**	Green Cap	**71**	Free Gill attach
21	Woods (habitat)	**47**	Purple Cap	**72**	White Gill
22	Waste (habitat)	**48**	White Cap	**73**	stem-height
23	Urban (habitat)	**49**	adnate gill attach	**74**	pores gill attach
24	Paths (habitat)	**50**	Decurrent Gill attach	**75**	stem-width
25	Meadows (habitat)				

Table 7: Feature mapping by indices

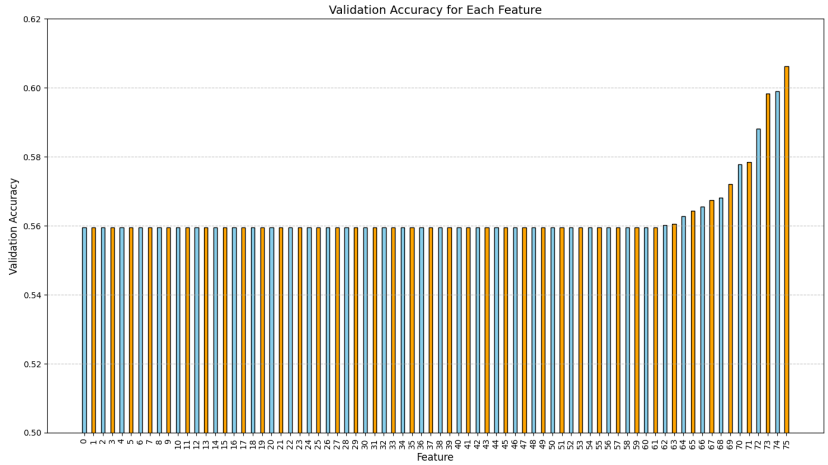

Figure 13: Val Accuracy Individual Features

The one piece of information we get from these tables are that the stem color is the least important and the stem-width is the most important, relatively.

5.2 Iteratively Removing Un-important Features

After iteratively removing features, it seemed like the validation accuracy just kept going down, which was an interesting result. This could indicate that most all the features contribute to the lower validation accuracy. There is not one feature that is clearly most important. All the features taken together indicate whether a mushroom is poisonous or edible. [**See 7 for the index mapping**]

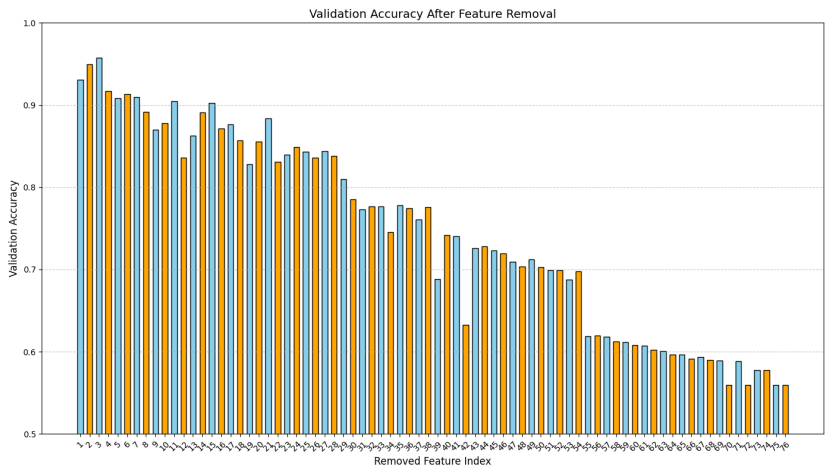

Figure 14: Val Accuracy After Feature Removal

6 Conclusion

In this project, I was able to build a neural network model to accurately predict the result of a binary classification problem: whether a given mushroom is poisonous or edible, given several different characteristics of the mushroom. In this project, I tested several different neural network models with varying layers and depth.

The most important and difficult part of this process was the pre-processing of the data. In this stage, it took me a while to figure out that the 20% data missing was the right threshold to drop a column. I also attempted to use label encoding instead of one-hot-encoding on the categorical features, but label encoding reduced the model's accuracy significantly.

After running the models, the final best model ended up being 4-neurons in the third layer, 2-neurons in the second layer, and 1-neuron in the first layer. This model led to accuracies upward of 94% for both the training and validation sets.

After finding the best model, I ran a model of the individual features, as an attempt to find the most important feature. The next part of the analysis was to iteratively remove features, starting with the most unimportant feature, which yielded an interesting result: as I removed the features, the validation accuracy kept falling until it was near the random baseline accuracy. The process exemplified that there were no single feature that was clearly correlated with the edibility of a mushroom.These features had to be taken together for the best result.